

**Carnegie
Mellon
University**

Grey-Box Program Analysis: Runtime Prediction and Beyond

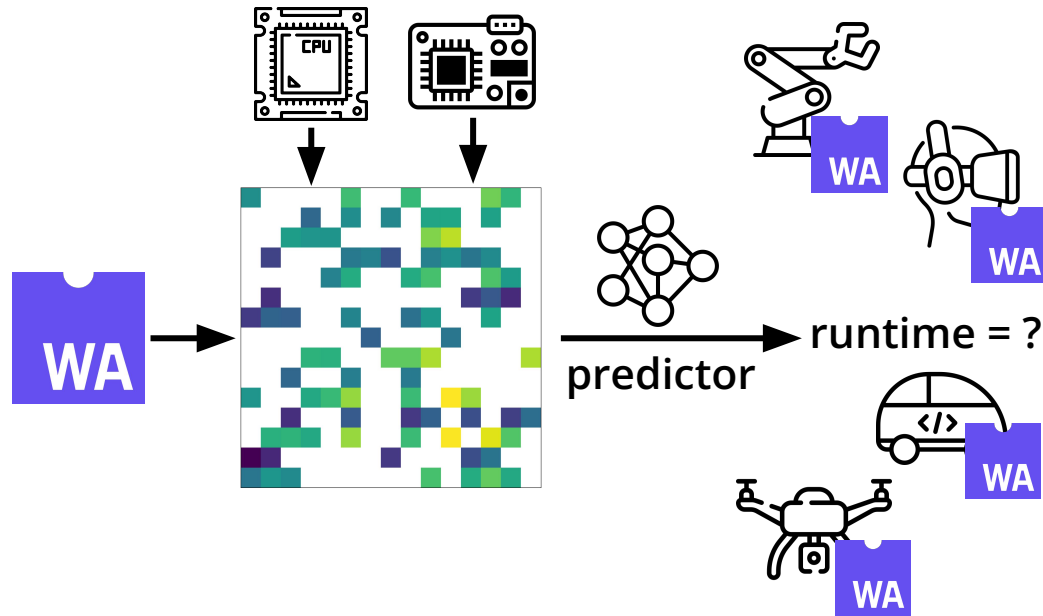
Tianshu Huang

tianshu2@andrew.cmu.edu



BOSCH

Runtime Prediction



Runtime Prediction

Static Analysis with **per-device model**

- Very tricky (halting problem)
- Hard to predict average case

Dynamic Analysis, with **device transfer model**

- Invasive instrumentation required
- Different model for each pair of devices

Dynamic Analysis, with **simulator-device transfer model**

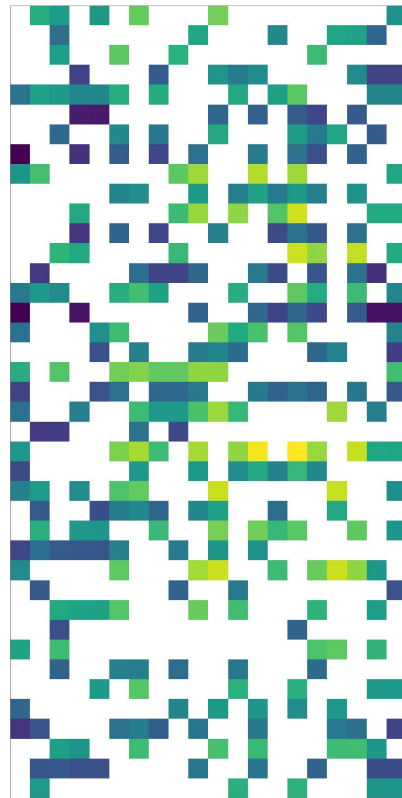
- Need to build a simulator for each device

Matrix Completion

Resource Prediction is **Matrix Completion**

- Rows $\times$ Columns:
Workloads $\times$ Devices
- Observe runtime of a subset of (module, device) pairs, predict unobserved entries

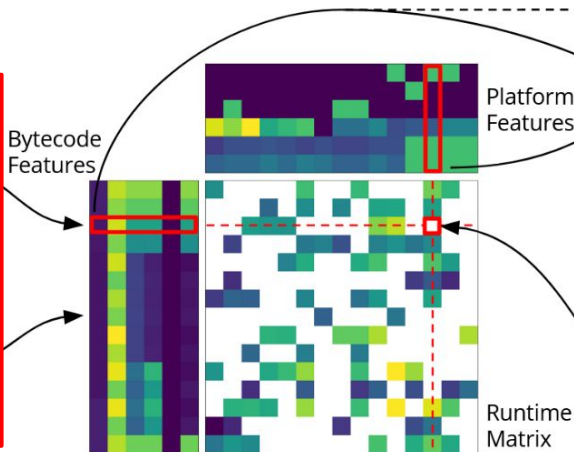
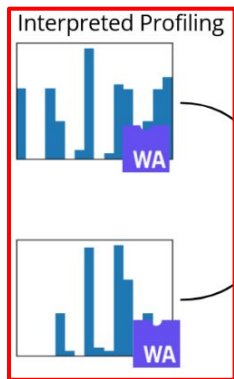
Workloads



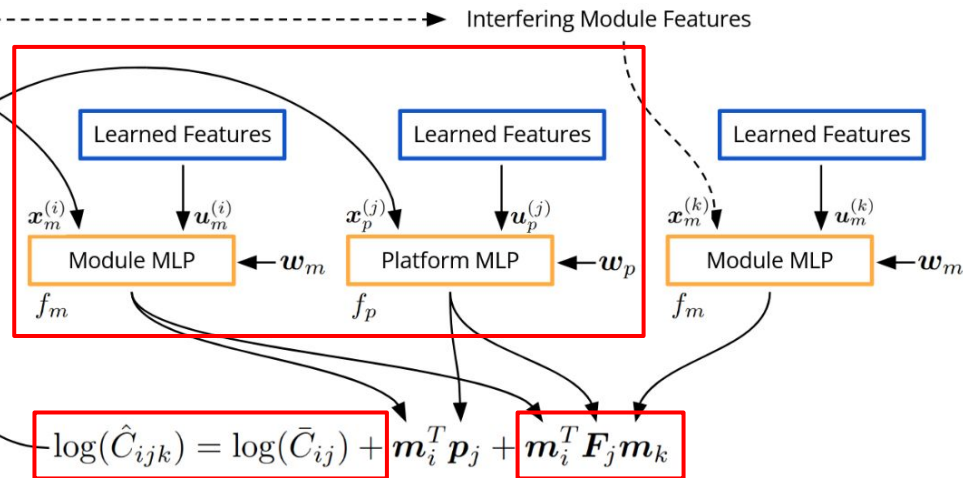
Devices

Method: **Matrix Factorization** (with Side Information)

1. Bytecode Features



3. "Two Tower" Model



2. Log-Residual Objective

4. Interference Term

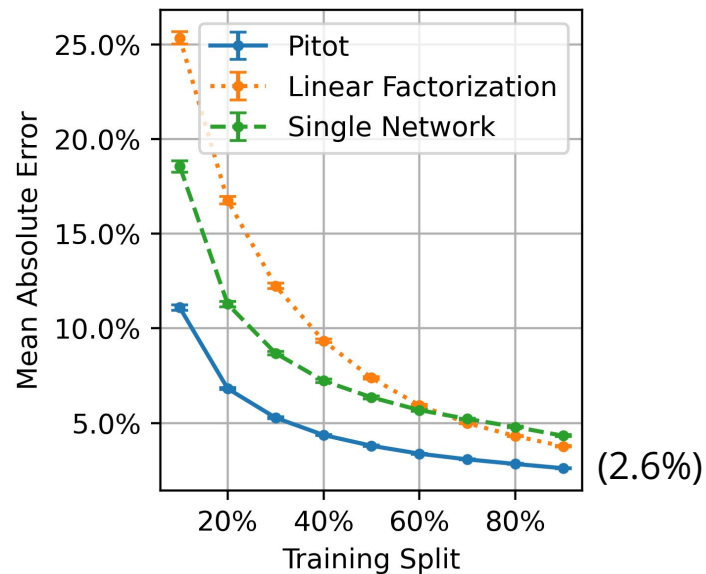
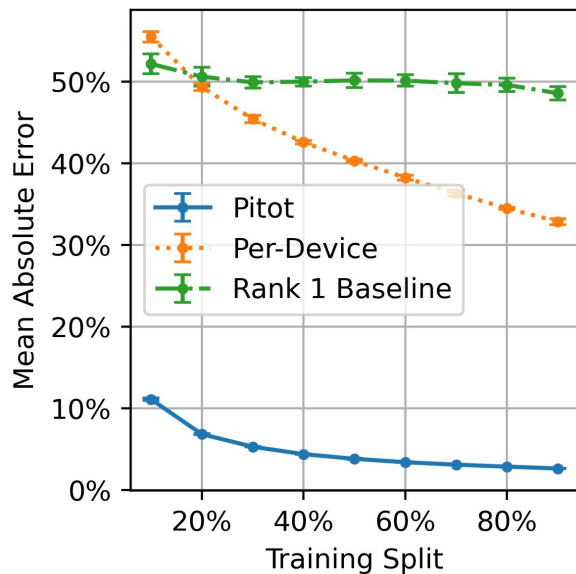
Dataset: N=30062

- **24 unique devices:** x86 Intel & AMD, ARM A & M class, and RISC-V
- **8 Wasm runtimes:** interpreted, JIT, & AOT
- **176 benchmarks** from <1ms to 30s long

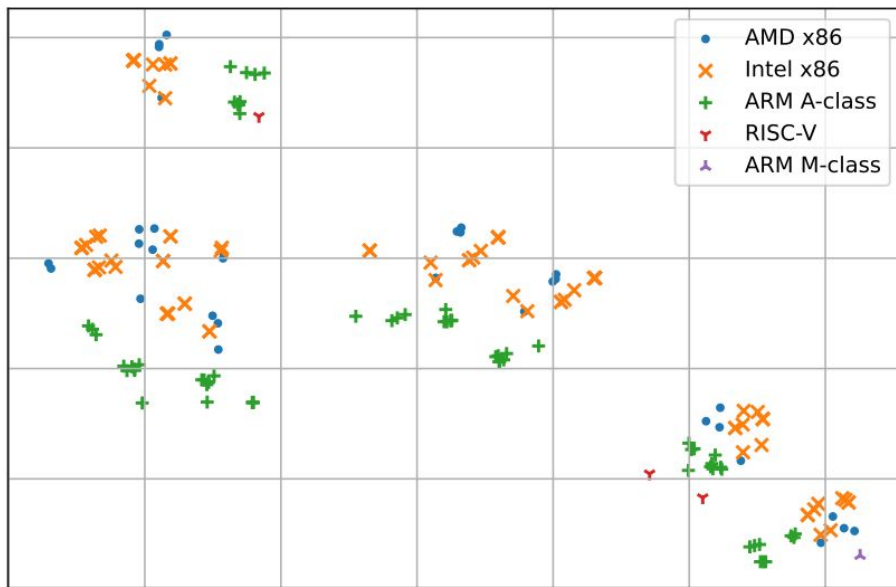


It works

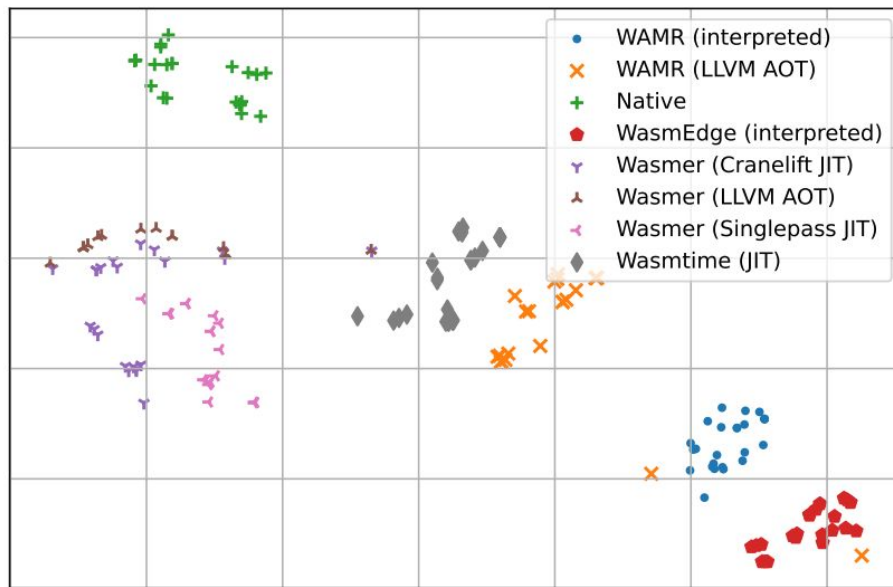
Pitot vs Baselines:



Characterizing Platforms*



(a) Platforms by Microarchitecture Type

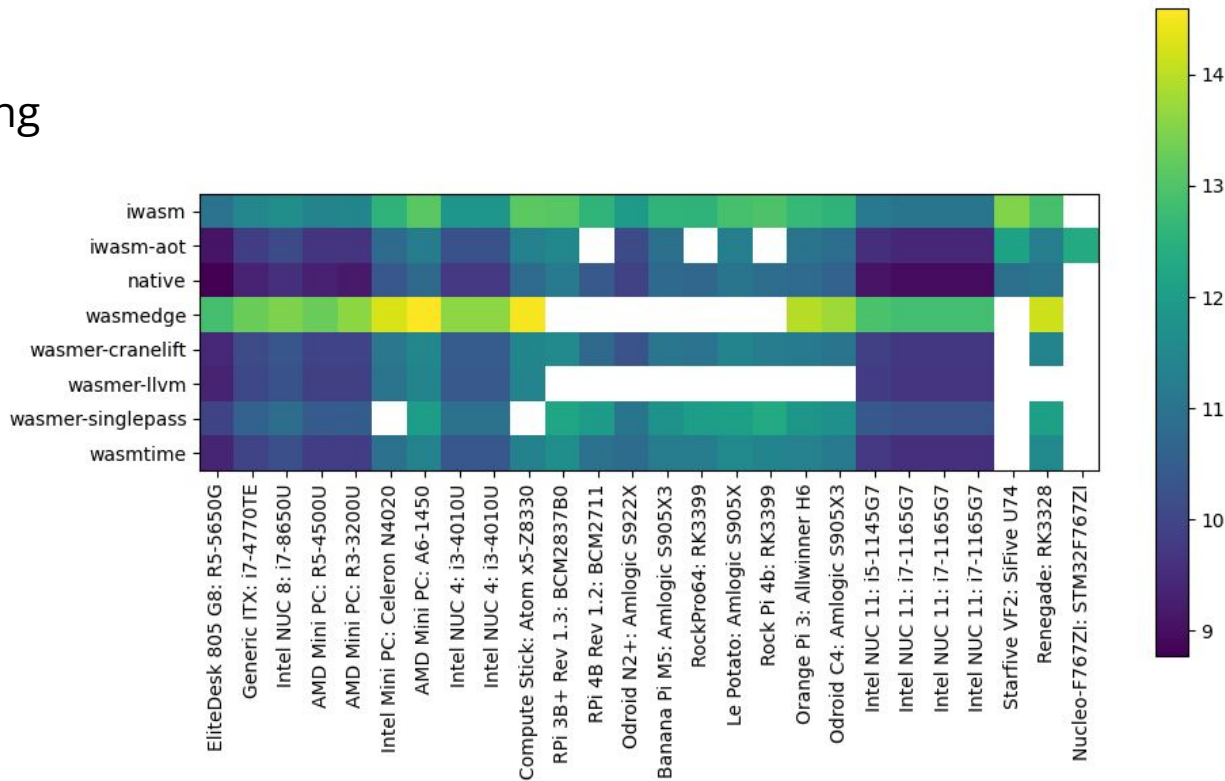


(b) Platforms by WebAssembly Runtime

*TSNE projection into 2 dimensions of the learned embeddings

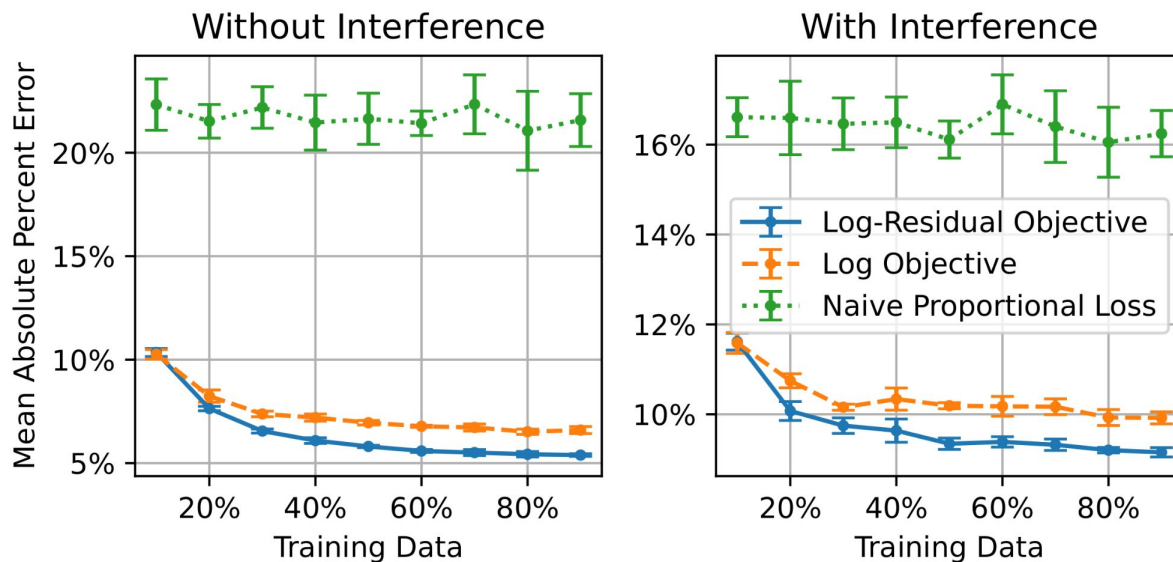
Log-Residual Objective

- Geometric Averaging
= Log Arithmetic Averaging
- Residual Objective
= Normalize for scalar “speed” / “difficulty” first



Aside: Why Log Runtime?

- Central limit theorem doesn't apply if random variables are “too” correlated ...
- ... but what happens when we take the log?



The Big Problem: **Data Dependence**



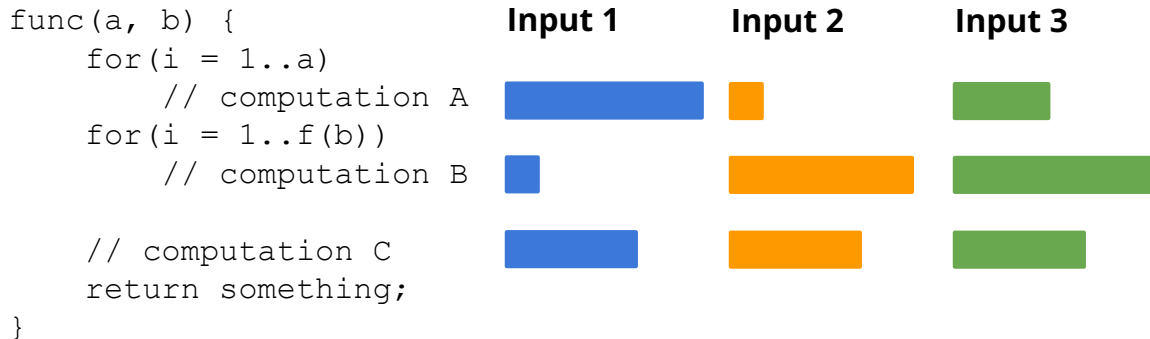
Motivation: **Code Coverage Instrumentation**

The problem:

- compute can **vary greatly with inputs**
- **can't understand** the input data

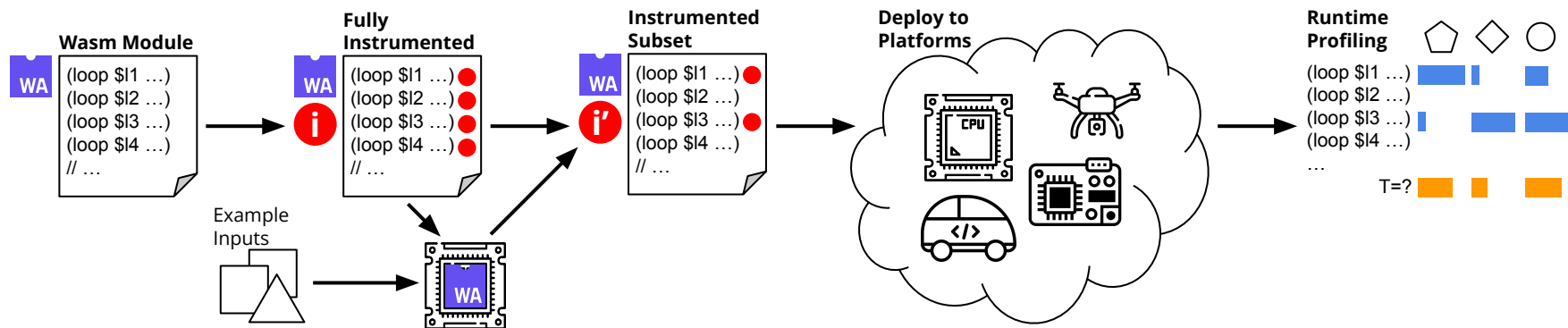
Our solution:

- **code frequency = input data** in all the ways that matter



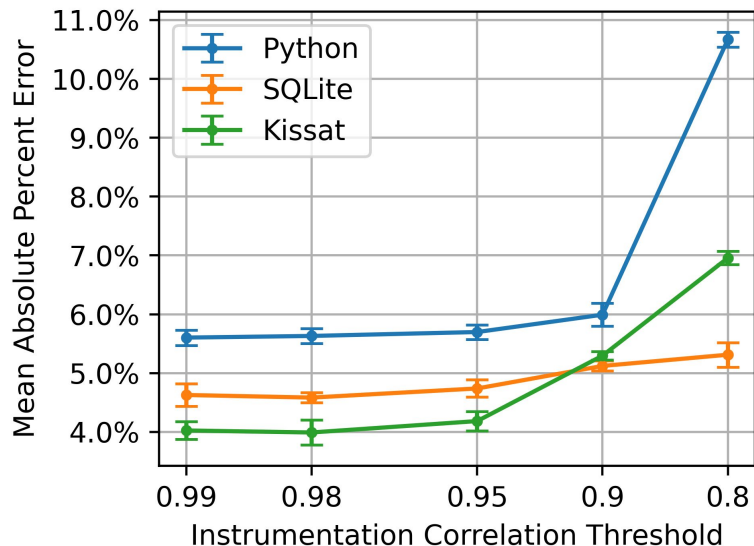
Our Approach: **Code Frequency via Loop Counts**

- Instrument loops
- Remove highly correlated loops
- More looping = more overhead
⇒ remove in decreasing order of # loops

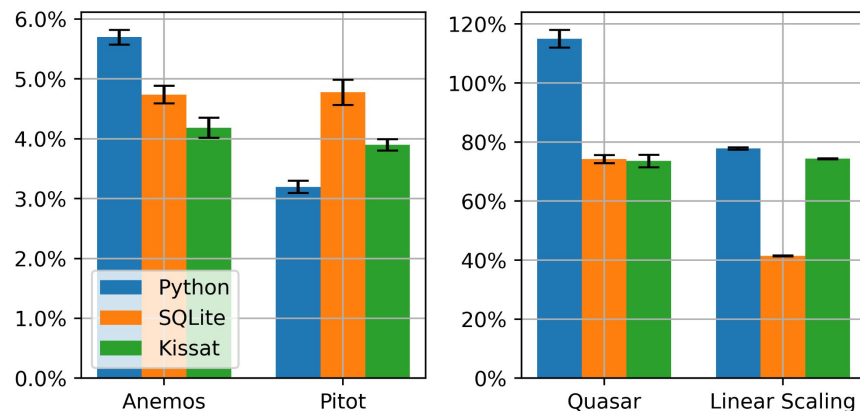


Results: Prediction Accuracy

- Can remove instrumentation with correlation >0.95



- Better than black box baselines
- Almost as good as a full opcode count



Beyond Runtime Prediction:

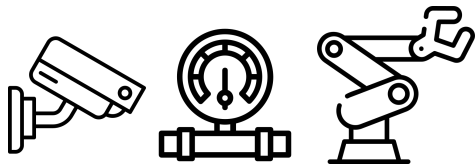
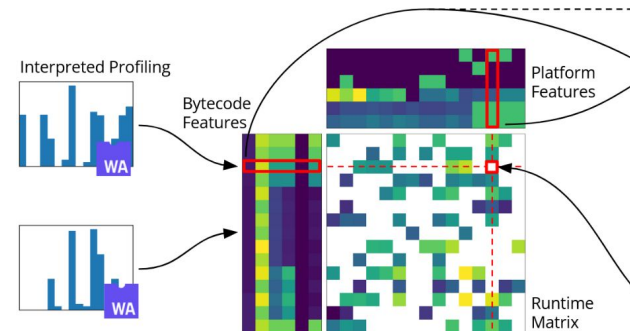
Program Embeddings

What is a “program embedding”?

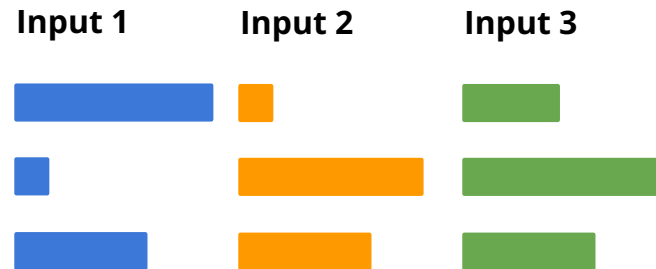
What can we do with it?

Program Embeddings

1. A behavior which can be measured
2. Some features which are correlated with this behavior in some way
3. An abstract vector space which captures this behavior



```
func(a, b) {  
  for(i = 1..a)  
    // computation A  
  for(i = 1..f(b))  
    // computation B  
  
  // computation C  
  return something;  
}
```



Program Embeddings

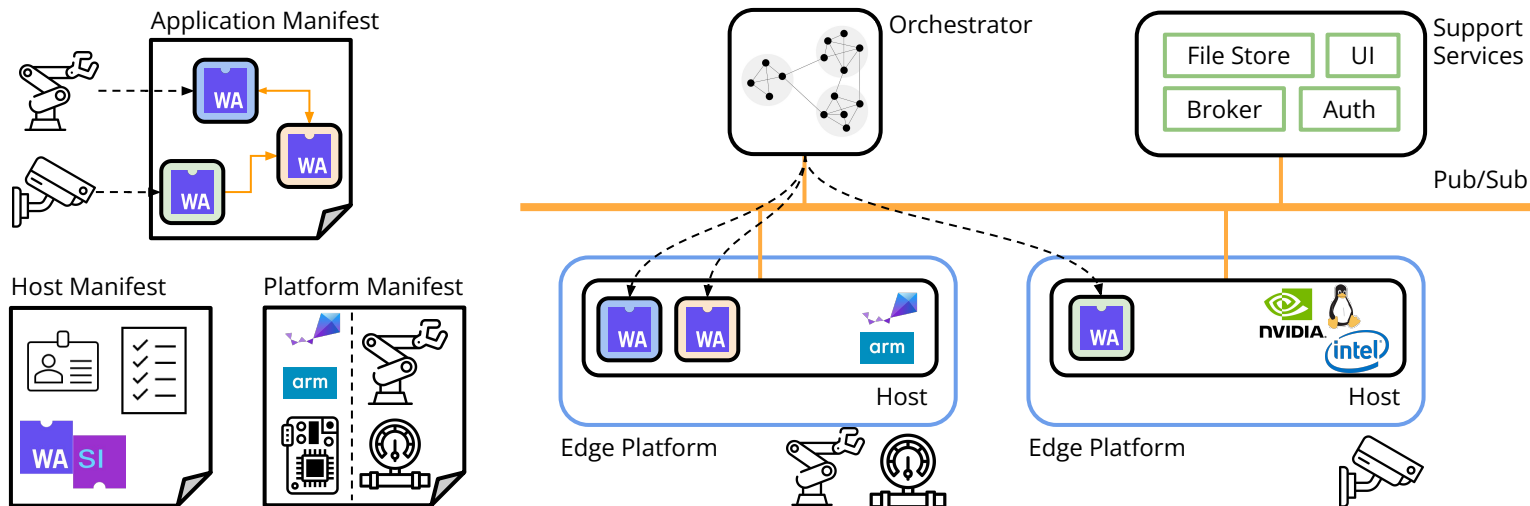
Possible **behaviors**:

- A specific program state, *as specified by the user*
- The digital output of a program, *as compared to past outputs*
- The physical output of a machine, *as interpreted by a ML model*
- ?

Possible **features**:

- “Code coverage”
- Memory state
- API calls / Syscalls / I/O traces
- Compute profile
- Program communication on a bus
- ?

Silverline



Challenges

Data

... it's mostly just data